

Building a Financial Data Normalisation Engine

A Practical Guide to Architecture, AI Integration, and Cloud Deployment

PE CoPilot Project

February 2026

1. What This Guide Covers

This document walks through the process of building a cloud-based system that takes messy financial reports from multiple companies and turns them into clean, comparable data. The kind of reports that arrive as Excel spreadsheets, CSV exports, and PDF board packs, each formatted differently, with different labels, and different accounting conventions.

The system described here was built for a private equity fund manager who oversees several portfolio companies. Each company uses a different accounting system (Sage, Xero, QuickBooks) and submits reports in a different format. The fund manager needs to compare performance across all companies using a single set of standardised metrics.

While the specifics are tailored to private equity, the architectural patterns apply to any scenario where you need to ingest heterogeneous data, extract structured information using AI, validate the outputs, and present them through a dashboard.

Who is this for?

Anyone who wants to understand how a modern data normalisation system is built, from the database and cloud infrastructure, through to AI integration and quality assurance. You do not need to be a software engineer to follow along, though some familiarity with cloud services helps.

2. The Problem We Are Solving

A private equity fund manages a portfolio of companies. Each quarter (or month), those companies submit financial reports. The challenge is that every company reports differently:

- **Different labels:** One company calls it "Net Sales", another calls it "Total Revenue", a third calls it "Turnover". They all mean the same thing.
- **Different formats:** Company A sends an Excel workbook with three tabs. Company B sends a single CSV export from their accounting system. Company C sends a PDF board pack with tables buried across several pages.
- **Different conventions:** Some companies record costs as negative numbers, others as positive. Some include formulas that show up blank when exported. Some report monthly, others quarterly.

Without automation, a fund analyst would manually open each file, find the relevant numbers, enter them into a master spreadsheet, and try to spot trends or problems. This is slow, error-prone, and does not scale.

The system we build automates this entire process: upload a file, extract the numbers, map them to a standard set of metrics, validate the output, and display everything on a dashboard.

3. Architecture: How the Pieces Fit Together

The system is built on Google Cloud Platform (GCP) using a set of services that each handle one job well. Here is the high-level flow:



3.1 Why Google Cloud Platform?

GCP was chosen for several practical reasons:

- **Firestore** is a document database that works well for semi-structured data. Financial metrics vary by company, so a rigid SQL schema would be limiting. Firestore lets each company record store different fields without complex migrations.
- **Cloud Storage (GCS)** handles file uploads reliably, with built-in versioning and access controls. Raw files are preserved as an audit trail.
- **Cloud Run** runs the application as a container. It scales to zero when idle (saving money) and scales up automatically under load. There are no servers to manage.
- **Pub/Sub** decouples file upload from processing. When a file is uploaded, a message is published. A separate process picks it up and runs the normalisation pipeline. This means uploads are fast (the user does not wait for AI processing) and failures in processing do not affect the upload.
- **Cloud Scheduler** runs periodic jobs, like checking which companies have not submitted reports recently and creating reminder tasks.

3.2 Why Python and FastAPI?

Python is the natural choice for a project that combines data parsing (pandas, openpyxl, pdfplumber) with AI integration (Anthropic Claude API). FastAPI provides a modern, async web framework with automatic API documentation, request validation, and high performance.

Pydantic models define the shape of every data structure in the system, from API requests to database records to AI responses. This means type errors are caught at the boundary, not deep inside business logic.

3.3 The Processing Pipeline

The heart of the system is a multi-layer processing pipeline. Each layer has a specific job:

Layer	Name	What It Does
Layer 1	Extraction	Opens the raw file and extracts text. Uses openpyxl for Excel, pandas for CSV, pdfplumber for PDF. Pure code, no AI.

Layer 1.5	Calculation	Applies company-specific formulas to fill gaps. For example, if "Gross Profit" is blank but "Revenue" and "Cost of Sales" exist, it computes the value. Deterministic, no AI.
Layer 2	AI Normalisation	Sends the extracted text to Claude (Anthropic's AI) along with the company's metric schema and mapping instructions. Claude returns structured JSON with the canonical metrics.
Layer 3	Validation	Checks for missing required metrics and calculates period-over-period variances (percentage changes from the last submission).
Layer 3.5	Sanity Checks	Deterministic financial validation: sign constraints (revenue must be positive), accounting identities (gross profit cannot exceed revenue), magnitude bounds, and variance spike detection.
Summary	AI Summary	A faster, cheaper AI model writes a 3–5 sentence executive summary, flags risks, and suggests follow-up actions.

Why multiple layers?

Each layer is independently testable and replaceable. If a better PDF parser emerges, you swap Layer 1 without touching the AI. If the AI model improves, Layer 2 changes but validation stays the same. This separation also means deterministic code handles what it can (cheaper, faster, predictable), and AI only handles the genuinely ambiguous parts.

4. AI Integration: Using LLMs Responsibly

The system uses Anthropic's Claude models for two distinct tasks, deliberately choosing different models based on how much accuracy matters for each.

4.1 Model Tiering

Not all AI tasks require the same level of capability. Using the most powerful model for everything is like hiring a senior accountant to make photocopies. The system routes tasks to the appropriate model:

Task	Model	Why This Model	Cost Impact
Financial extraction	Claude Sonnet (larger)	High stakes: wrong numbers mean wrong decisions	Higher cost, worth it
Summarisation	Claude Haiku (smaller)	Writing prose from structured data is less demanding	~40–50% cheaper

The key insight is that financial data extraction from messy, inconsistent spreadsheets is genuinely hard. The AI needs to understand that "Net Sales" at one company means the same thing as "Turnover" at another, that negative numbers might mean costs, and that a blank cell next to a formula label means the value needs to be inferred. This is where you want the best model.

Summarisation, by contrast, receives clean, structured numbers as input. It is writing prose from a table. A smaller, faster, cheaper model handles this reliably.

4.2 Prompt Engineering

The AI receives carefully structured prompts that include:

- **The company's metric schema:** a list of the nine canonical metrics the fund tracks (revenue, gross profit, EBITDA, net income, cash balance, total debt, net assets, operating cashflow, headcount).
- **Company-specific mapping instructions:** notes like "Revenue is labelled Net Sales in the Sage export" or "Costs are stored as negative numbers".
- **The raw extracted text:** the output of Layer 1, capped at a sensible length to control costs.

The AI is instructed to return only valid JSON, to never fabricate numbers, and to express its confidence as a score between 0 and 1. Low confidence triggers a human review workflow.

4.3 Guarding Against AI Errors

AI models can hallucinate (invent plausible-sounding but incorrect data), make unit errors, or misinterpret ambiguous labels. The system defends against this at multiple levels:

1. **Confidence scoring:** The AI self-reports its confidence. Updates below 0.5 confidence are automatically flagged for human review.
2. **Missing metric detection:** If more than two required metrics are absent, the update is flagged.
3. **Deterministic sanity checks (Layer 3.5):** Pure code validates the AI's output against financial rules:
 - Revenue and gross profit must be positive (a negative revenue is almost certainly an error)
 - Gross profit cannot exceed revenue (by more than a 5% rounding tolerance)
 - EBITDA cannot exceed gross profit
 - No single metric should exceed £10 billion (catches pence-vs-pounds unit errors)
 - A 500%+ change from the previous period is flagged as a likely extraction error
4. **Variance tracking:** Period-over-period changes are calculated and shown on the dashboard. Unusual swings are visually highlighted.
5. **Human review workflow:** Flagged updates sit in a "needs review" state until a human confirms or corrects them.

The principle

Use AI where it adds value (interpreting messy, inconsistent data), but verify its output with deterministic code wherever possible. Every number the AI produces passes through a gauntlet of sanity checks before it reaches the dashboard.

5. Data Flow: From Upload to Dashboard

5.1 File Upload

When someone uploads a file, three things happen in a specific order designed to be safe if anything goes wrong:

1. **Deduplication check:** The system computes a SHA-256 hash (a digital fingerprint) of the file's contents. It checks whether this exact file has been uploaded before for this company. If it has, the upload is rejected with a clear message pointing to the existing record. This catches duplicate submissions regardless of what the file is named.
2. **Cloud Storage upload:** The raw file is saved to Google Cloud Storage, preserving an audit trail. If this fails, nothing else happens.
3. **Database record creation:** A record is created in Firestore with status "pending". If this fails, the Cloud Storage file is automatically cleaned up (transaction safety).
4. **Processing trigger:** A message is published to Pub/Sub to trigger asynchronous processing. If this fails, the upload still succeeds. The pending record can be reprocessed manually later.

This design means the user gets a fast response (the upload is confirmed in under a second) and processing happens in the background. The event-driven architecture also means the system can handle many uploads concurrently without blocking.

5.2 The Dashboard

The dashboard shows a portfolio-level view: a card for each company displaying its health status (green, amber, or red), the latest metrics, a summary, and any flagged risks. Clicking a card opens a detailed view with the full history of submissions, metric trends over time, and pending tasks.

Health status is calculated automatically based on three factors: how recently the company submitted data, how large the variances are, and how many metrics are missing. Configurable thresholds determine the green/amber/red boundaries.

5.3 Tasks and Staleness

The system automatically creates tasks when companies go too long without submitting data. A Cloud Scheduler job runs periodically and checks each company's last submission date. If it exceeds the configured threshold, a task is created prompting someone to follow up.

Tasks can also be created manually for ad-hoc follow-ups, like "Verify the revenue figure with the CFO" or "Request a resubmission of the Q4 pack".

6. Testing and Quality Assurance

The system has over 220 automated tests that run in seconds. These are not just a checkbox exercise; they are the safety net that allows the codebase to evolve without breaking existing functionality.

6.1 What We Test

Test Category	Count	What It Covers
Calculator (Layer 1.5)	~60	Formula computation for each company (gross profit, EBITDA, etc.), sign handling (negative costs vs positive), parsing edge cases
Parsers (Layer 1)	~30	Excel extraction (multi-sheet, formula cells), CSV encoding detection, PDF table extraction across pages
Normalisation pipeline	~25	End-to-end pipeline with mocked AI responses, error handling, status determination logic
Financial validators	~21	Sign constraints, accounting identities, magnitude bounds, zero-revenue checks, variance spike detection
Deduplication	~7	Content hash matching, same content different names, different content same names, company scoping
API and integration	~40	HTTP endpoints, authentication, transaction safety (GCS cleanup on Firestore failure), Pub/Sub event handling
Storage and infrastructure	~20	GCS retry logic, URL parsing, Pub/Sub publishing, Cloud Scheduler callbacks, model tiering
Dashboard and detail views	~20	Portfolio aggregation, company detail endpoint, metrics history ordering, health scoring

6.2 How Tests Work

Most tests use "mocking", which means they replace external services (the database, cloud storage, the AI) with controlled substitutes. This means tests run in under two seconds without needing any cloud credentials or internet access. The AI calls are replaced with pre-built JSON responses so we can test how the system handles every possible output.

Each new feature is built alongside its tests. When we added deduplication, we wrote tests proving that identical files are rejected, different files pass through, and the check is scoped to the correct company. When we added financial validation, we wrote tests for every rule: negative revenue, gross profit exceeding revenue, extreme variances, and so on.

7. What Comes Next

The system as built handles the core workflow: upload, extract, normalise, validate, and display. Several natural extensions are planned:

7.1 Planned Components

Feature	Description	Infrastructure
Email ingestion	Accept reports via email. A dedicated inbox receives attachments, parses the email metadata, and feeds files into the same processing pipeline.	SendGrid inbound parse, Cloud Functions
Digest reports	Weekly or monthly email summaries sent to the fund manager, summarising activity across all portfolio companies, highlighting anything that needs attention.	Cloud Scheduler, SendGrid outbound
Google Sheets export	Push normalised data to a shared Google Sheet, maintaining a live view that non-technical stakeholders can access without logging into the app.	Google Sheets API, service account
PDF report generation	Generate formatted PDF reports for board meetings or LP reporting, including charts and tables.	WeasyPrint or ReportLab
OCR support	Handle scanned PDFs (images of documents) by running optical character recognition before extraction.	Google Document AI or Tesseract
Multi-currency	Support portfolio companies that report in different currencies, with automatic conversion to the fund's base currency.	Exchange rate API, currency metadata

7.2 Infrastructure That Grows With It

Each new feature follows the same pattern: a new service module, a new router for API endpoints, new models for data structures, and tests for everything. The event-driven architecture means new processing steps can be added without modifying existing ones. For example, email ingestion would publish the same Pub/Sub event as the upload endpoint, reusing the entire processing pipeline.

As the number of portfolio companies grows, Firestore scales automatically. As processing volume increases, Cloud Run adds more instances. The only component that requires careful cost management is the AI layer, which is why model tiering was implemented early.

8. Summary

Building a financial data normalisation engine involves solving several interconnected problems: parsing diverse file formats, mapping inconsistent labels to standard metrics, validating AI-generated output, and presenting actionable insights through a dashboard.

The key architectural decisions are:

- **Separate what code can do from what AI must do.** Deterministic parsing, calculation, and validation handle the predictable parts. AI handles the genuinely ambiguous parts (mapping "Net Sales" to "revenue").
- **Verify AI output with rules.** Every number the AI produces passes through financial sanity checks. This is cheaper, faster, and more reliable than asking the AI to check its own work.
- **Use the right model for the job.** Expensive models for high-stakes extraction, cheaper models for prose generation. This cuts costs without sacrificing accuracy where it matters.
- **Prevent duplicates at the boundary.** Content-based hashing catches duplicate uploads regardless of filename, before any processing resources are spent.
- **Design for failure.** Every external operation (storage, database, messaging) has explicit error handling and cleanup. The upload flow is transactional: if step 2 fails, step 1 is rolled back.
- **Test everything.** Over 220 automated tests cover every layer, from individual formula calculations to end-to-end API behaviour. Tests run in under two seconds and require no cloud credentials.

PE CoPilot — Built with Python, FastAPI, Google Cloud Platform, and Anthropic Claude